UNIVERSITY OF ŽILINA

FACULTY OF MANAGEMENT SCIENCE AND INFORMATICS

Semestrálna práca 3 — Dokumentácia

Diskrétna simulácia

Author: Kyrylo Raiskyi
Group name: 5ZIA22

Academic year: 2025/2026
Summer term

1. Zadanie

V tejto semestrálnej práci analyzujem a modelujem prevádzku pohotovostného oddelenia, ktoré už dlho zápasí s problémom dlhých čakacích dôb. Hlavnou úlohou je otestovať a navrhnúť personálnu štruktúru, ktorá zabezpečí hladký chod nemocnice aj pri obmedzených zdrojoch. Celá simulácia začína vytvorením dvoch nezávislých vstupných tokov (pacienti prichádzajúci sami a pacienti privezení sanitkou). Títo pacienti prechádzajú procesom triage a následne počiatočným vyšetrením, kde im je na základe ich stavu pridelená priorita liečby od 1 do 5. Z logiky modelu je od začiatku jasné, že dynamika systému vo veľkej miere závisí od tohto triage, ktoré následne určuje poradie v hlavnej čakárni na lekárske ošetrenie.

Systém neustále a aktívne riadi kapacitné obmedzenia: počet dostupných sestier, lekárov a vyšetrovacích miestností. To je zvlášť zrejmé zo spätnej väzby pri vchode, kde pacienti prichádzajúci sanitkou automaticky predbiehajú tých, ktorí prichádzajú sami, čo vytvára veľké prekážky a čakacie rady priamo pri dverách. Kľúčovou podmienkou a cieľom celej štúdie je simulovať 28-dňové cykly a nájsť personálne obsadenie, pri ktorom systém dokáže ošetriť najkritickejších pacientov (priorita 1) do 15 minút a všetkých ostatných do maximálne pol hodiny.

2. Spracovanie údajov

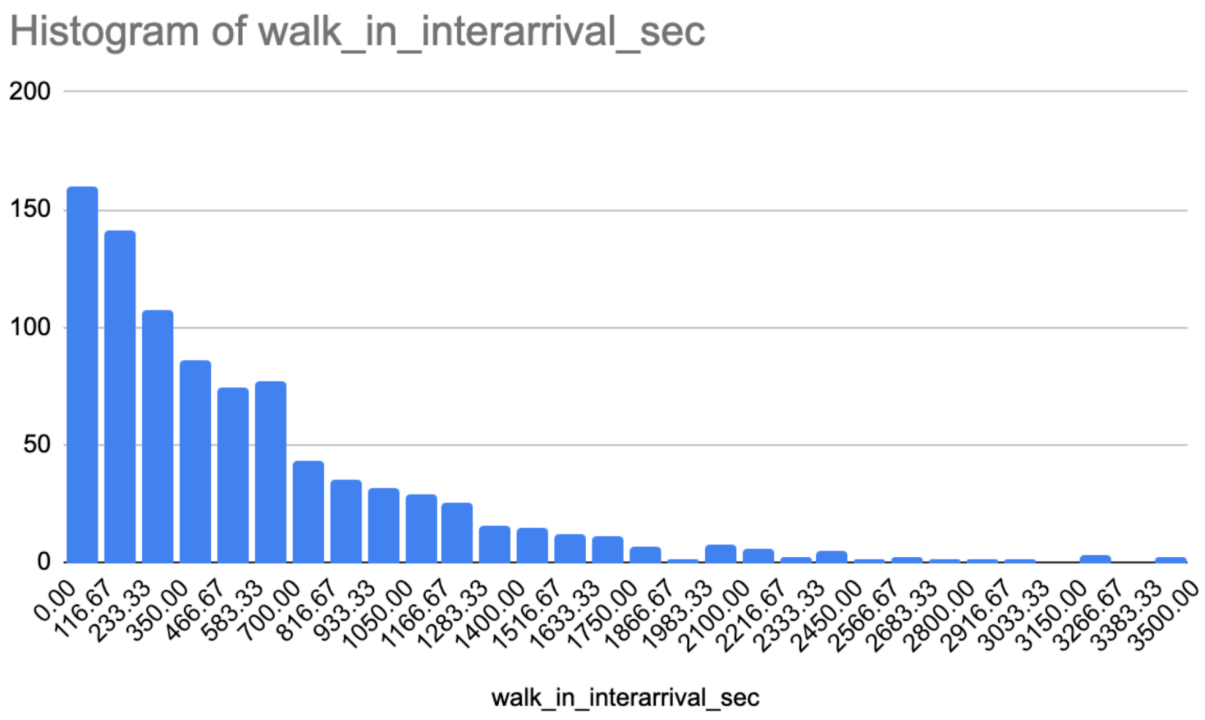
V prvom rade som sa analyzoval dostupné údaje o príchodoch pacientov som vygeneroval skript v jazyku Python, ktorý údaje rozdelil do dvoch tabuliek podľa spôsobu príchodu a vypočítal intervaly medzi jednotlivými príchodmi. Výsledky ukázali, že dva nezávislé vstupné toky majú odlišné štatistické rozdelenia.

```
df = pd.read_csv('arrivals.csv', sep=';', index_col=False, names=['datum', 'cas',
'prichod_sanitkou', 'extra'], skiprows=1)
df = df.drop(columns=['extra'])
df['datetime'] = pd.to_datetime(df['datum'] + ' ' + df['cas'])
df = df.sort_values('datetime')

amb_df = df[df['prichod_sanitkou'] == 1].copy()
walk_df = df[df['prichod_sanitkou'] == 0].copy()
amb_times = amb_df['datetime'].diff().dt.total_seconds().dropna().reset_index(drop=True)
walk_times = walk_df['datetime'].diff().dt.total_seconds().dropna().reset_index(drop=True)

out_df = pd.DataFrame({
   'ambulance_interarrival_sec': pd.Series(amb_times),
   'walk_in_interarrival_sec': pd.Series(walk_times)
```

```
})
out_df.to_csv('interarrival_times.csv', index=False)
print("Data saved to interarrival_times.csv")

def fit_distributions(data, name):
   distributions = {
       'exponential': ('expon', st.expon),
       'uniform': ('uniform', st.uniform),
       'triangular': ('triang', st.triang)
   }

   best_dist = None
   best_p = -1
   best_params = None

   print(f"\n--- Fitting distributions for: {name} ---")

   for friendly_name, (scipy_name, dist) in distributions.items():
       params = dist.fit(data)
       D, p_value = st.kstest(data, scipy_name, args=params)
       print(f"{friendly_name.capitalize()}: KS-statistic={D:.4f}, p-value={p_value:.4f}")

       if p_value > best_p:
           best_p = p_value
           best_dist = friendly_name
           best_params = params

   if best_p < 0.05:
       print(f"\n>> No theoretical distribution fit well (Best p-value={best_p:.4f}).
Defaulting to EMPIRICAL.")

       counts, bin_edges = np.histogram(data, bins='auto')
       probabilities = counts / counts.sum()
       print("\nParameters for your Empirical Generator (Histogram/Discrete):")
       print("You can input these probability weights and bin bounds into your simulation
program.")
       for i in range(len(counts)):
           print(f"Bin {i+1}: Range [{bin_edges[i]:.2f} to {bin_edges[i+1]:.2f}] sec ->
Probability: {probabilities[i]:.4f} ({probabilities[i]*100:.2f}%)")

   else:
       print(f"\n>> Best fitting distribution for {name}: {best_dist.capitalize()}
(p-value={best_p:.4f})")

       if best_dist == 'exponential':
           loc, scale = best_params
           print(f"Parameters: Mean (1/Lambda) = {scale:.2f}")
       elif best_dist == 'uniform':
           loc, scale = best_params
           print(f"Parameters: Min = {loc:.2f}, Max = {(loc + scale):.2f}")
       elif best_dist == 'triangular':
           c, loc, scale = best_params
           print(f"Parameters: Min = {loc:.2f}, Mode = {(loc + c*scale):.2f}, Max = {(loc +
scale):.2f}")

fit_distributions(walk_times, "Walk-in Arrivals")
fit_distributions(amb_times, "Ambulance Arrivals")
```

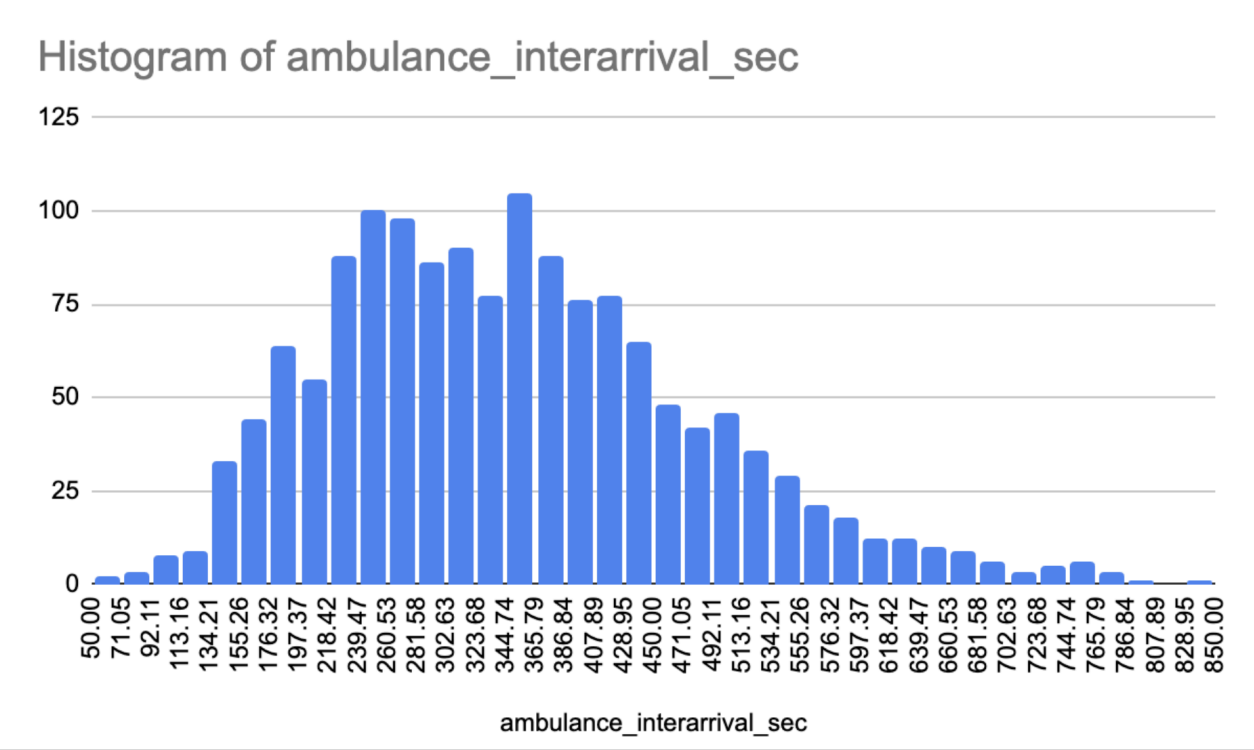
Úryvok kódu č.1 – vygenerovaný Python skript

Python skript vypísal potrebné výsledky do konzoly a navyše som údaje analyzoval vo forme histogramu, v prípade pacientov prichádzajúcich pešo, ktorý vykazoval klasickú exponenciálnu krivku smerom nadol. Táto domnienka bola jednoznačne potvrdená Kolmogorov-Smirnovovým testom (p-hodnota 0,9824). Pre tento tok bolo zvolené exponenciálne rozdelenie s priemernou hodnotou 573,99 sekúnd.



Obrázok č.1 – histograma pacientov prichádzajúcich pešo, intervaly medzi jednotlivými príchodmi

Pri pohľade na údaje o pacientoch prichádzajúcich sanitkou je viditeľná pomerne jasná krivka v tvare zvonu s vrcholom uprostred, čo vylučuje použitie exponenciálneho rozdelenia. Na základe KS-testu však žiadne zo štandardných teoretických rozdelení nezodpovedá údajom dostatočne presne (všetky vrátili hodnotu p rovnú 0). Z tohto dôvodu bolo empirické rozdelenie určené ako najvhodnejšie. Pre tento tok bude model používať empirický generátor vytvorený z 25 definovaných intervalových skupín a ich zodpovedajúcich pravdepodobnostných váh v rozmedzí od 56 do 830 sekúnd.



Obrázok č.2 – histograma pacientov prichádzajúcich sanitkou, intervaly medzi jednotlivými príchodmi

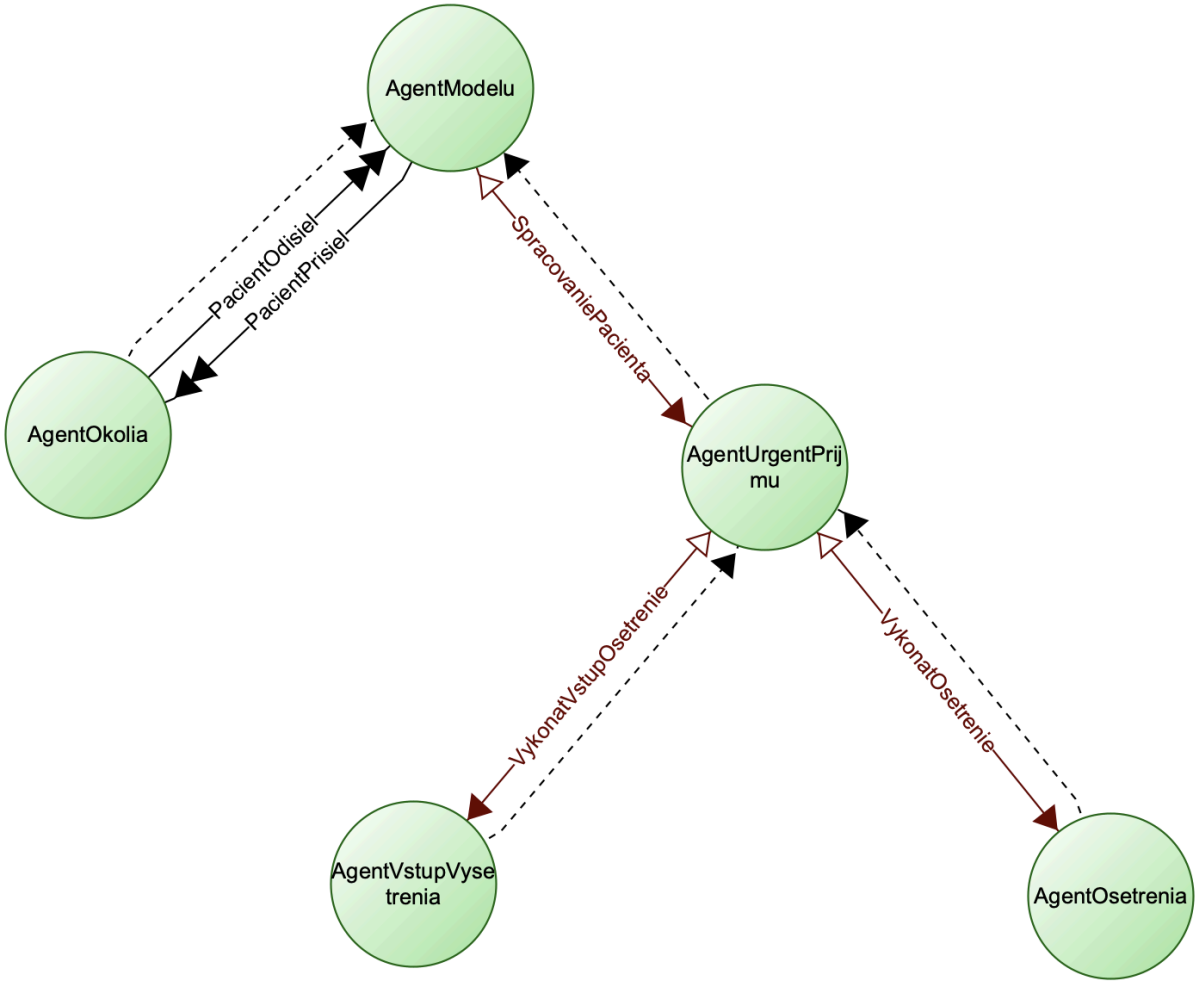
Výstup generovaného Python skriptu na konzole je uvedený nižšie:

```
Data saved to interarrival_times.csv

--- Fitting distributions for: Walk-in Arrivals ---
Exponential: KS-statistic=0.0153, p-value=0.9824
Uniform: KS-statistic=0.5388, p-value=0.0000
Triangular: KS-statistic=0.3521, p-value=0.0000

>> Best fitting distribution for Walk-in Arrivals: Exponential (p-value=0.9824)
Parameters: Mean (1/Lambda) = 573.99

--- Fitting distributions for: Ambulance Arrivals ---
Exponential: KS-statistic=0.2819, p-value=0.0000
Uniform: KS-statistic=0.2930, p-value=0.0000
Triangular: KS-statistic=0.1175, p-value=0.0000

>> No theoretical distribution fit well (Best p-value=0.0000). Defaulting to EMPIRICAL.
```

Obrázok č.3 – výstup na konzole

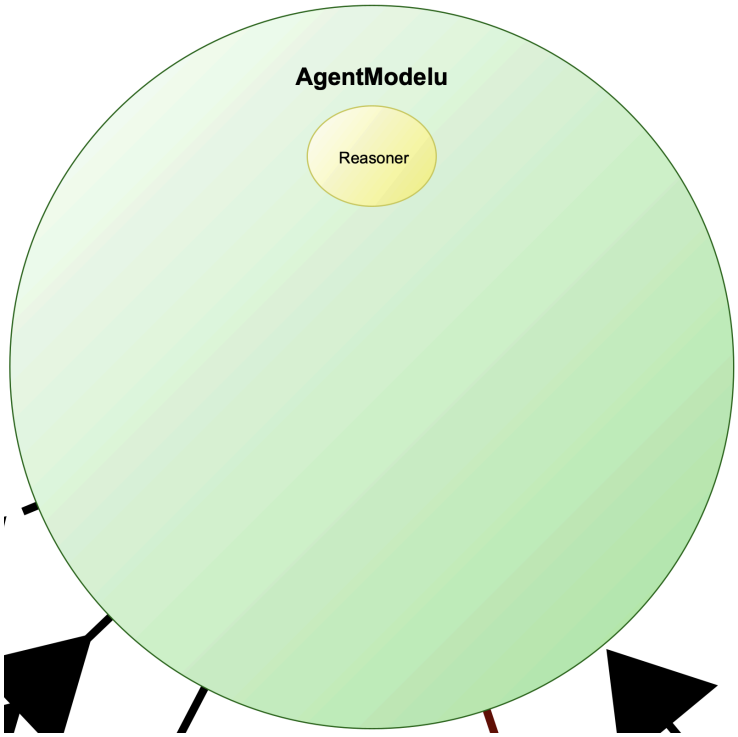
3. Návrh systému

Agentova hierarchia bola vytvorená v programe ABABuilder a je zobrazená nižšie.



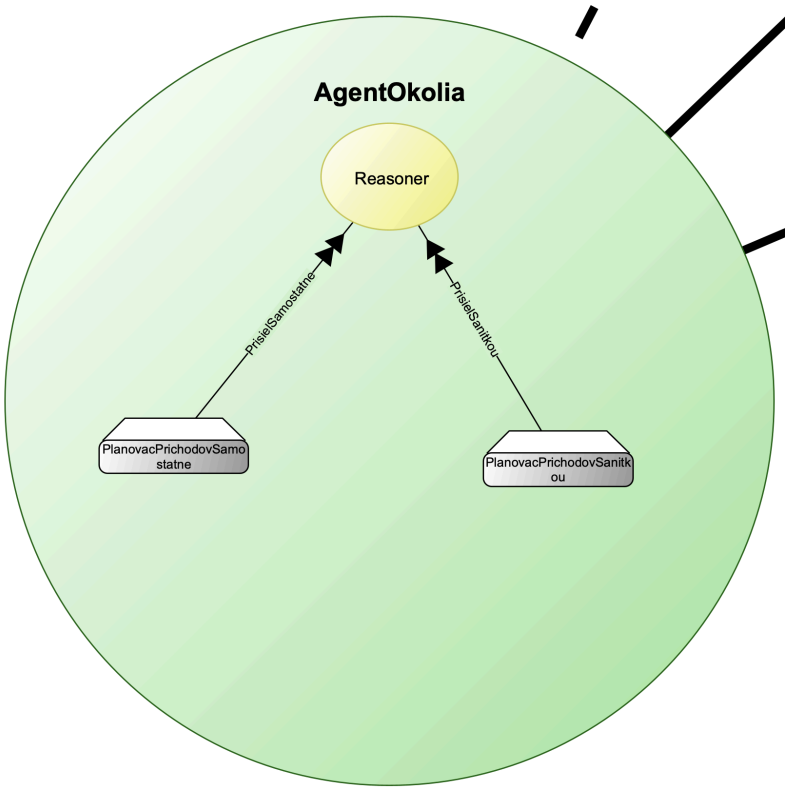
Obrázok č.4 – agentova hierarchia

AgentModelu je najvyšší koreňový agent simulácie, ktorý zahŕňa celý nemocničný systém. V súčasnosti využíva iba predvolený modul Reasoner na smerovanie správ medzi vonkajším prostredím a pohotovosťou, takže neobsahuje žiadne špecifické procesy ani plánovače.



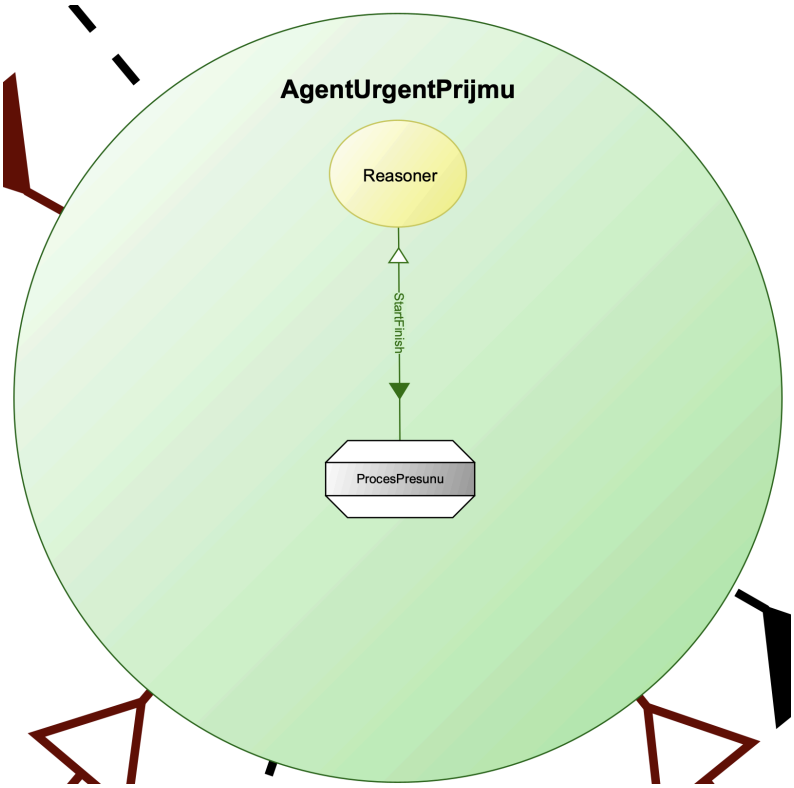
Obrázok č.5 – AgentModelu

Tento agent reprezentuje vonkajší svet a zodpovedá za získavanie nových pacientov. V rámci systému som vytvoril dva nezávislé plánovače (PlanovacPrichodovSamostatne a PlanovacPrichodovSanitkou), ktoré využívajú matematické generátory typu Exponential a Empirical na to, aby do systému priebežne vkladali príchody pacientov ktorí prichádzajú pešo a sanitkou.



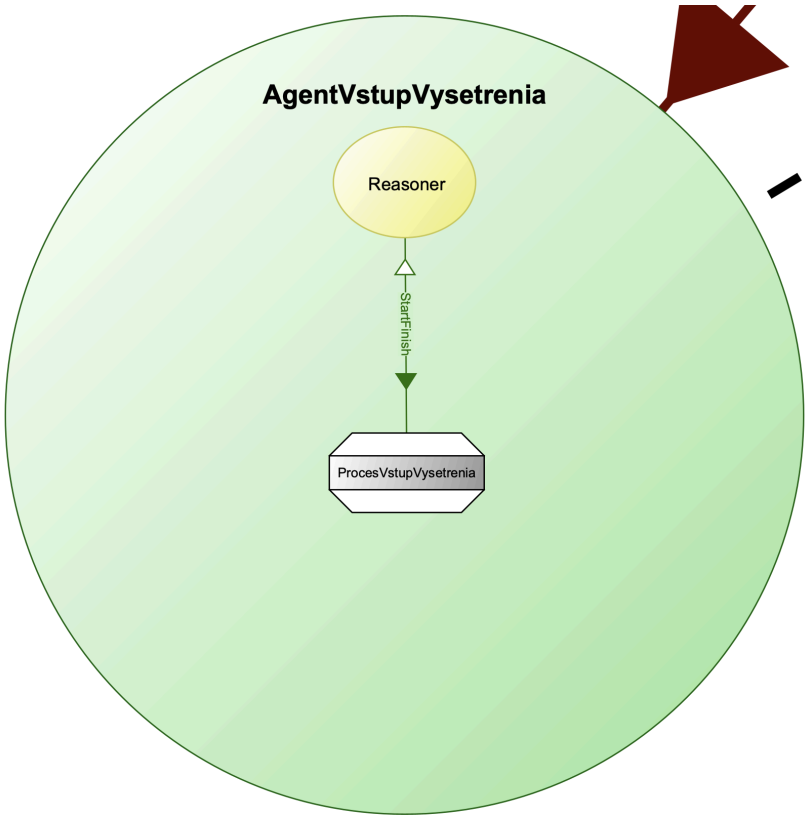
Obrázok č.6 – AgentOkolia

Tento agent riadi tok pacientov medzi jednotlivými oddeleniami. Obsahuje funkciu s názvom ProcesPresunu, ktorá na krátku chvíľu zadrží pacientov, aby sa simulovalo, ako fyzicky prechádzajú chodbou, než sa dostanú k do cieľa.



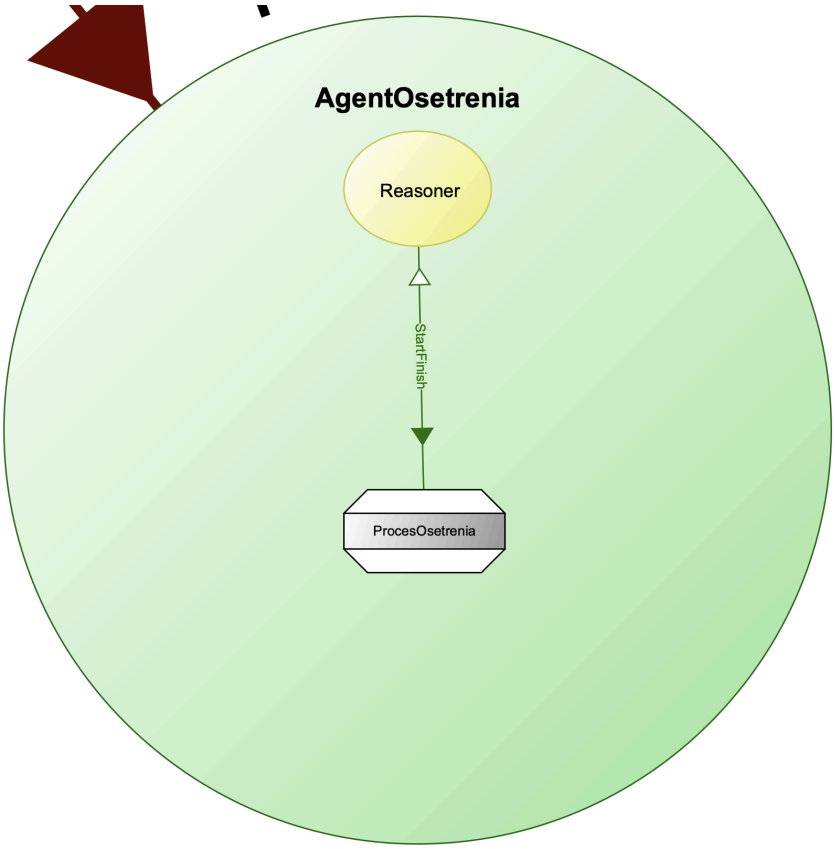
Obrázok č.7 – AgentUrgentPrijmu

Tento agent zabezpečuje vstupné vyšetrenie a posúdenie stavu pacientov. Jeho hlavnou úlohou je prijímať prichádzajúcich pacientov, posúdiť ich stav a priradiť im zdravotnú prioritu v rozmedzí od 1 do 5, než ich nasmeruje do hlavnej čakárne.



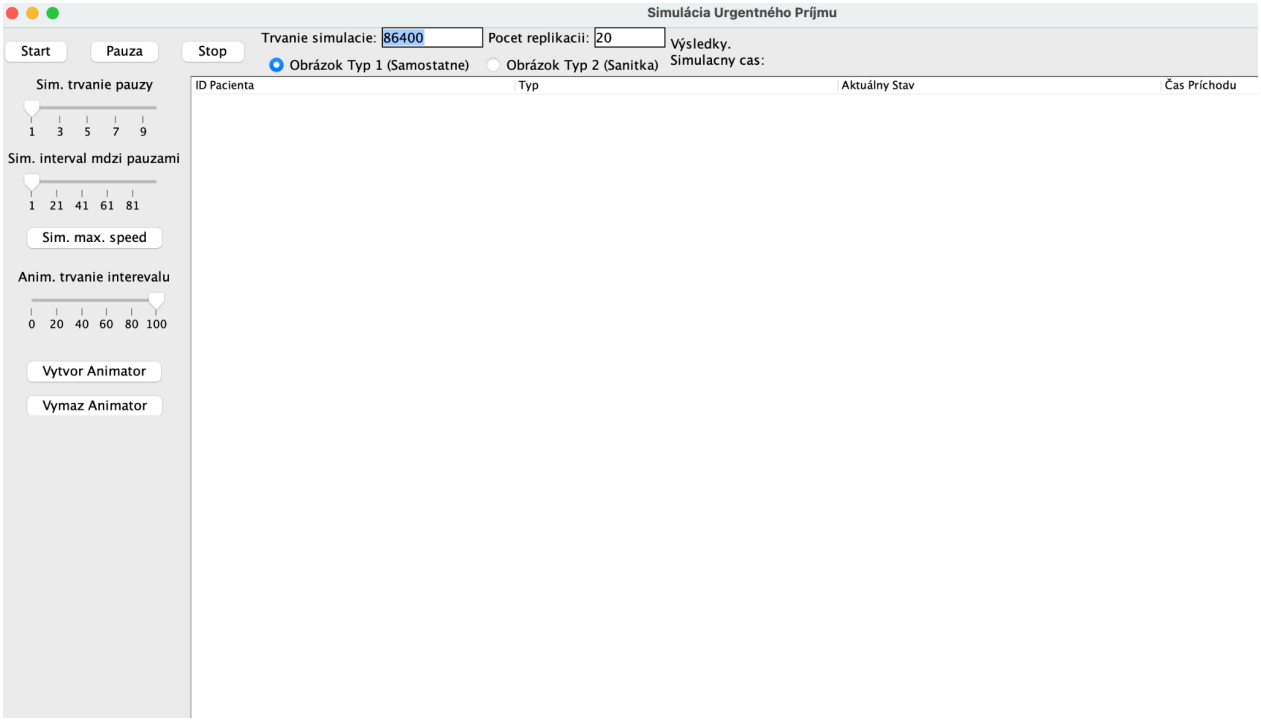
Obrázok č.8 – AgentVstupVysetrenia

AgentOsetrenia je jadro nemocnice, kde prebieha lekárska starostlivosť. Je zodpovedné za riadenie lekárov a spravovanie poradí podľa priority, pričom zabezpečuje, aby pacienti v kritickom stave boli vyvedení z čakárne a ošetrení ako prví



Obrázok č.9 – AgentOsetrenia

4. GUI aplikácie



Obrázok č.10 – GUI aplikácie

Na vizuálne sledovanie logiky simulácie som vytvoril grafické používateľské rozhranie, ktoré sleduje priebeh vykonávania a aktuálny stav systému. V hornej časti sa nachádzajú ovládacie prvky pre vykonávanie a vstupné parametre (ako napríklad čas simulácie a počet opakovaní), pričom vľavo sú posuvníky na reguláciu rýchlosti, ktoré slúžia na spomalenie alebo zrýchlenie simulácie.

V súčasnosti systém sleduje entity v reálnom čase. Obsahuje tabuľku, ktorá zobrazuje ID každého aktívneho pacienta, spôsob príchodu, aktuálny stav a čas príchodu. Tento vizuálny prehľad umožňuje jasne analyzovať tok pacientov bez objednávky a sanitiek a overiť, čo sa v systéme deje v danej sekunde.